FitTracker

Technical Architecture Specification, SRS Specification, and Mathematical Model of Automatic Progressive Overload

Lead Software Architect & Tech Lead

August 14, 2026

Abstract

This document serves as the formal software engineering specification for the mobile system **FitTracker**. Engineered under a *Local-First* paradigm and modular *Clean Architecture*, FitTracker automates resistance training management using an adaptive progressive overload engine based on mathematical 1RM ($1 Repetition Maximum$) estimation formulas and RPE/RIR autoregulation. It provides the Software Requirements Specification (SRS), UML use cases, SQLite database design, and the progressive algorithm mathematical model.

Contents

1 Introduction and Objectives

1.1 Context and Rationale

In resistance training and muscle hypertrophy, the principle of **progressive overload** serves as the core foundation for driving long-term physiological adaptations. However, the vast majority of mobile applications on the market currently suffer from two critical limitations:

1. Strict reliance on Internet connectivity and cloud infrastructure (*Cloud-First*), which degrades the user experience in gyms with weak coverage or underground facilities.
2. Lack of deterministic automatic progressive overload algorithms based on physiological variables and quantitative metrics such as estimated 1RM and the RPE/RIR scale (*Rating of Perceived Exertion / Repetitions in Reserve*).

FitTracker is designed to solve both limitations through a **Local-First** mobile architecture powered by an embedded SQLite engine and an adaptive algorithmic progression model.

1.2 System Objectives

1.2.1 General Objective

Design, architect, and implement a cross-platform mobile application (React Native + Expo in TypeScript) characterized by 100% offline operation and Clean Architecture, capable of calculating and automatically increasing workload and volume for athletes based on historical performance.

1.2.2 Specific Objectives

- Implement an ultra-fast local persistence layer using SQLite, ensuring zero network latency and fault tolerance.
- Develop the mathematical 1RM ($1RepetitionMaximum$) estimation module by averaging classic Epley and Brzycki models with RIR correction.
- Provide a modular system architecture strictly separated into Domain, Use Cases, Repositories, and UI layers.
- Deliver formal academic technical documentation for engineering audit and research.

1.3 Acknowledgments and Data Sources

We formally acknowledge developer **Hasan Yıldırım** for creating and maintaining the open-source repository `exercises-dataset`. His structured JSON dataset provides the complete exercise database, metadata, and media elements powering the offline local seeding system in our persistence layer.

1.4 Artificial Intelligence Assistance Statement

This software project, including system architectural design, mathematical progression model formulation, embedded SQLite persistence layer, TypeScript source code, Jest unit test suite, and formal LaTeX technical documentation, was developed in pair-programming collaboration with **Antigravity**, an AI Coding Assistant developed by Google DeepMind.

2 Software Requirements Specification (SRS)

This specification adapts the IEEE 830 standard for offline-first mobile systems.

2.1 Functional Requirements (FR)

Table 1: System Functional Requirements

Code	Name	Description
FR-01	Exercise Catalog Management	The system shall allow creating, modifying, and categorizing exercises by primary and secondary muscle groups.
FR-02	Routine Configuration	The system shall allow defining routines composed of exercise blocks, target rep ranges, and desired RPE.
FR-03	Live Set Logging	Allows logging weight (kg), reps (r), and RPE/RIR (R) in real-time with an automated rest timer.
FR-04	Automated Overload Calculation	The system shall compute weight/rep recommendations for the next session upon workout completion.
FR-05	1RM Estimation	The system shall compute current and historical estimated 1RM per session using validated mathematical models.
FR-06	Offline Nutrition Tracking	Daily logging of macronutrients (protein, carbs, fats) and total calories without requiring external APIs.
FR-07	Body Metrics Tracking	Tracking body weight, body fat percentage, and girth measurements.

2.2 Non-Functional Requirements (NFR)

Table 2: System Non-Functional Requirements

Code	Name	Metric / Acceptance Criteria
NFR-01	Offline Operability	100% of read and write operations must execute locally without an internet connection.
NFR-02	Write Performance	Inserting a completed set into SQLite must execute in under $50\,ms$.
NFR-03	Portability	The React Native (TypeScript) codebase must be portable across Android and iOS without logic modifications.
NFR-04	Data Integrity	SQLite database must use ACID transactions to prevent data corruption during unexpected app closures.

3 UML Use Cases Specification

3.1 System Actors

- **Athlete / End User:** Individual executing workout routines, logging sets, managing rest timers, and analyzing overload analytics offline.

3.2 System Use Case Diagram

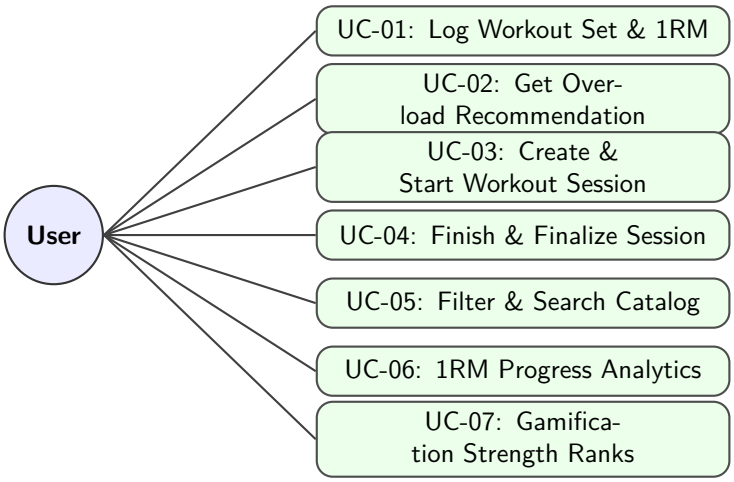

Figure 1: UML Use Case Overview for FitTracker Domain Architecture.

3.3 Detailed Specifications

3.3.1 UC-01: Log Workout Set and Compute Progression

Use Case	**UC-01: Log Workout Set and Compute Automatic Progression**
Primary Actor	Athlete / End User
Preconditions	At least one exercise exists in the catalog and an active workout session is started.
Main Flow	1. The user selects the current exercise in the active session. 2. The system displays target weight (kg) and reps calculated by UC-02. 3. The user inputs load, completed reps, and RPE ($6.0 - 10.0$). 4. The user confirms set registration. 5. The system calculates estimated 1RM, persists the set to SQLite, and updates session effective volume.
Alternative Flow	*3a. User marks set as Warmup Set:* - System records the set but excludes it from effective volume and 1RM calculation.
Postconditions	Set recorded in SQLite with calculated 1RM.

3.3.2 UC-02: Get Automatic Overload Recommendation

3.3.3 UC-03: Create & Start Workout Session

3.3.4 UC-04: Finish & Finalize Workout Session

3.3.5 UC-05: Filter & Search Exercise Catalog

3.3.6 UC-06: Get Exercise Historic Metrics & 1RM Evolution

Use Case	UC-02: Get Automatic Overload Recommendation
Primary Actor	Athlete / End User
Preconditions	Exercise selected from catalog.
Main Flow	1. System fetches last working sets logged for the exercise. 2. System calculates average RIR and completion rate against target rep range. 3. System applies load rule ($+2.5\,kg$ compound / $+1.25\,kg$ isolation / $MAINTAIN$ / $DELOAD$). 4. System presents recommended load and target reps with mathematical justification.
Postconditions	Recommendation generated deterministically without side effects.

Use Case	UC-03: Create & Start Workout Session
Primary Actor	Athlete / End User
Preconditions	Application initialized with SQLite schema.
Main Flow	1. User enters session title (e.g., "Leg Day - Heavy") and optional notes. 2. System generates unique session identifier and records start timestamp. 3. System transitions session state to active.
Postconditions	New session record persisted in SQLite database.

Use Case	UC-04: Finish & Finalize Workout Session
Primary Actor	Athlete / End User
Preconditions	Active workout session in progress.
Main Flow	1. User taps "Finish Workout". 2. System calculates session duration, total tonnage, and total effective volume. 3. System identifies any Personal Records (PRs) set during the session. 4. System updates session metadata and displays workout summary.
Postconditions	Session state marked complete; summary metrics stored.

Use Case	UC-05: Filter & Search Exercise Catalog
Primary Actor	Athlete / End User
Preconditions	SQLite exercise table populated with dataset.
Main Flow	1. User types search query or selects filter parameters (muscle group, equipment, type). 2. System executes SQL query with substring matching and filter constraints. 3. System returns sorted list of matching exercises.
Postconditions	Filtered exercise list displayed to user.

Use Case	UC-06: Get Exercise Historic Metrics & 1RM Evolution
Primary Actor	Athlete / End User
Preconditions	Exercise selected by user.
Main Flow	1. System queries SQLite for all historical sets of the target exercise. 2. System aggregates peak estimated 1RM per session date. 3. System calculates historical max weight, max volume, and total sets logged. 4. System returns time-series data suitable for charting.
Postconditions	Analytics DTO generated for visual rendering.

4 Architecture and Database Design

4.1 System Architecture (Clean Architecture)

FitTracker implements Clean Architecture structured by functional feature modules (*Feature-First*).

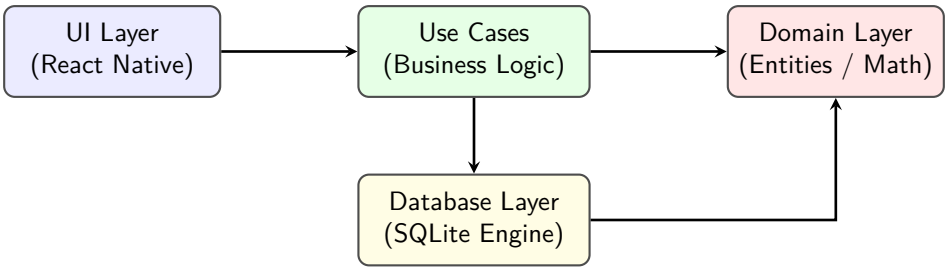

Figure 2: Layer Dependency Diagram in Clean Architecture.

4.2 Entity-Relationship Model and SQLite Schema

```sql
-- Exercises Table
CREATE TABLE exercises (
    id TEXT PRIMARY KEY NOT NULL,
    name TEXT NOT NULL,
    category TEXT NOT NULL,
    type TEXT NOT NULL,
    equipment TEXT NOT NULL,
    gif_url TEXT,
    instructions TEXT
);

-- Workout Sessions Table
CREATE TABLE workout_sessions (
    id TEXT PRIMARY KEY NOT NULL,
    name TEXT NOT NULL,
    date INTEGER NOT NULL,
    notes TEXT
);

-- Logged Exercise Sets Table
CREATE TABLE exercise_sets (
    id TEXT PRIMARY KEY NOT NULL,
    session_id TEXT NOT NULL,
    exercise_id TEXT NOT NULL,
    set_order INTEGER NOT NULL,
    weight_kg REAL NOT NULL,
    reps INTEGER NOT NULL,
    rpe REAL NOT NULL,
    is_warmup INTEGER NOT NULL DEFAULT 0,
    estimated_1rm REAL NOT NULL,
    FOREIGN KEY(session_id) REFERENCES workout_sessions(id) ON DELETE CASCADE,
    FOREIGN KEY(exercise_id) REFERENCES exercises(id) ON DELETE CASCADE
);
```

Listing 1: SQLite Relational DDL Schema

5 Automatic Progressive Overload Algorithm

5.1 Mathematical Formulations for 1RM Estimation

To quantify relative strength in a set of r repetitions executed with load w (expressed in kg) at a perceived exertion of $RPE \in [6.0, 10.0]$, we first calculate equivalent Repetitions in Reserve (RIR):

$$RIR = 10.0 - RPE \tag{1}$$

Estimated repetitions to absolute failure $r_{\max}$ are calculated as:

$$r_{\max} = r + RIR \tag{2}$$

We use a weighted combination of classic non-linear models by **Epley** and **Brzycki**:

1. **Epley Formula:**

$$1RM_{\text{Epley}} = w \cdot \left(1 + \frac{r_{\max}}{30}\right) \tag{3}$$

2. **Brzycki Formula:**

$$1RM_{\text{Brzycki}} = w \cdot \frac{36}{37 - r_{\max}} \tag{4}$$

The final estimated set 1RM ($1RM_{\text{est}}$) is computed by averaging both models:

$$1RM_{\text{est}} = \frac{1RM_{\text{Epley}} + 1RM_{\text{Brzycki}}}{2} \tag{5}$$

5.2 Effective Volume and Automatic Progressive Overload

The **Total Effective Volume** ($V_{\text{effective}}$) of a session for a given exercise, considering working sets only ($is\_warmup = 0$), is expressed as:

$$V_{\text{effective}} = \sum_{i=1}^{N} w_i \cdot r_i \cdot \alpha(RPE_i) \tag{6}$$

where $\alpha(RPE_i)$ represents the fatigue weighting factor:

$$\alpha(RPE) = \begin{cases} 1.0 & \text{if } RPE \geq 8.0 \\ 0.85 & \text{if } 7.0 \leq RPE < 8.0 \\ 0.70 & \text{if } RPE < 7.0 \end{cases} \tag{7}$$

5.3 Next Session Load Progression Rule

The recommended load w_{next} for the subsequent session is adjusted based on session average RIR_{avg} and target repetition range $[R_{\min}, R_{\max}]$ completion:

$$w_{\text{next}} = \begin{cases} w_{\text{current}} + \Delta w & \text{if } r_i \geq R_{\max}\ \forall i \quad \text{and} \quad RIR_{\text{avg}} \geq 1.5 \\ w_{\text{current}} & \text{if } R_{\min} \leq r_i < R_{\max} \quad \text{or} \quad 0.5 \leq RIR_{\text{avg}} < 1.5 \\ w_{\text{current}} \cdot (1 - \beta) & \text{if } RIR_{\text{avg}} < 0.5 \quad \text{(deload or excessive fatigue)} \end{cases} \tag{8}$$

where Δw is the standard weight increment ($2.5\,kg$ for compound lifts, $1.25\,kg$ for isolation exercises) and $\beta \in [0.05, 0.10]$ is the fatigue deload factor.

5.4 Gamification Strength Rank Formulation

To provide offline gamification without server dependencies, we calculate a normalized strength rank score S_{rank} based on relative bodyweight performance:

$$S_{\text{rank}} = \left(\frac{1RM_{\text{est}}}{BW} \right) \cdot \lambda_{\text{sex}} \cdot \mu_{\text{type}} \tag{9}$$

where BW represents body weight in kg, $\lambda_{\text{sex}} = 1.45$ for female athletes (1.0 for male), and $\mu_{\text{type}} = 2.2$ for isolation exercises (1.0 for compound lifts). The athlete's strength rank tier (Wood, Iron, Bronze, Silver, Gold, Platinum, Diamond) is assigned by mapped score thresholds $S_{\text{rank}} \in [0.0, 2.5]$.